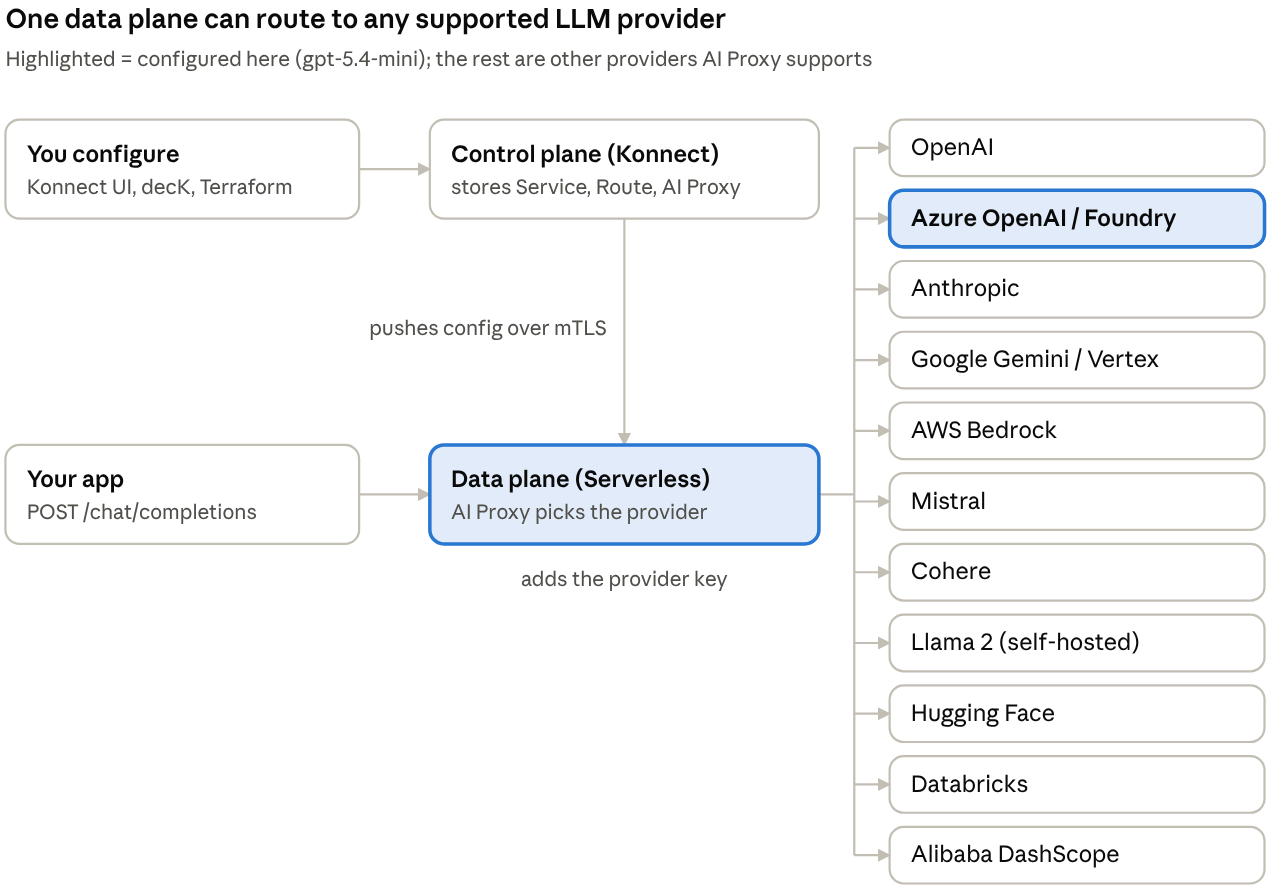
Kong Konnect AI Gateway with Azure AI Foundry — End-to-End Setup

Nov 10, 2026 · @Srinivasarao Polagani

Overview

Apps call a Kong **data plane** (the proxy URL). The data plane applies the config it gets from the **control plane** (Konnect) and forwards each request to the Azure AI Foundry deployment `gpt-5.4-mini`. Both planes are required; with a Serverless gateway Kong runs both.



architecture · config path and traffic path

The top row is where you make changes; the bottom row is where requests flow. Calling the control plane directly is what produced the mTLS error.

There are two ways to build this in Konnect:

Path	Where you configure	Status in this account
A. AI Proxy plugin on a Serverless API Gateway	API Gateway → Service → Route → AI Proxy plugin	Working: `https://4b1ed59e0a.us.serverless.gateways.konggateway.com/chat/completions`
B. AI Gateway 2.0 (Model Provider + AI Model)	AI Gateway → Providers, Models	Config saved, but no data plane was provisioned (proxy DNS returns NXDOMAIN)

Key values used throughout:

Item	Value
Konnect region	US (`us.api.konghq.com`)
Serverless control plane	`serverless-api-gateway-demo` (ID `5e42f6e0-73d1-4f2e-a9dc-bd210184514e`)
Azure resource (instance)	`srinipolagani-8291-resource`
Azure deployment	`gpt-5.4-mini` (Global Standard, 150 RPM / 150K TPM)
Azure API version	`2025-04-01-preview`
AI Gateway (Path B)	`AI-Gateway` , ID `35d895bb-08ce-48c3-ae48-00ffaea68dc1`

Prerequisites

Gather these before you start.

- [ ] **Konnect account** at cloud.konghq.com, US region. If you have two organizations, note which one you build in; entities in one are invisible from the other.
- [ ] **Azure AI Foundry resource** with a model deployed. In ai.azure.com → Models + endpoints, copy the deployment **Name** (here `gpt-5.4-mini`). Use the deployment name, not the model name.

- [ ] **Azure key**: Azure Portal → your resource → Keys and Endpoint → Key 1 or Key 2.
- [ ] **Resource name** from the endpoint: `https://<resource>.cognitiveservices.azure.com` → `<resource>` is the Azure instance.
- [ ] **Public network access** enabled on the Azure resource (Serverless gateways have no private connectivity).
- [ ] For config-as-code: a **Konnect personal access token** (profile → Personal access tokens) and decK (`brew install kong/deck/deck`).
- [ ] For testing: `curl` and optionally `jq` .

Step 1 — Create the control plane

The control plane stores your config; you manage it through the Konnect UI or decK and never send app traffic to it.

Path A — Serverless API Gateway

1. Konnect sidebar → **API Gateway** → **New** → **New API Gateway**.
2. Choose **Serverless**, enter a name (e.g. `serverless-api-gateway-demo`), click **Create**.
3. Open it → **Overview**. Copy the **Proxy URL** (`https://<id>.us.serverless.gateways.konggateway.com`). This is the only URL apps call.

Path B — AI Gateway 2.0

1. Konnect sidebar → **AI Gateway** → **New AI gateway**.
2. Choose deployment type **Serverless**, enter a name (e.g. `AI-Gateway`), create it.
3. Open it → **Connect** → copy the **Public edge DNS** address. That is its proxy URL.

Do not call the **Admin API** / `*.cp.konghq.com` address shown on the Overview page. It is the control plane endpoint and returns `mTLS headers missing` to normal clients.

Step 2 — Get a data plane

No data plane means no proxy URL and nothing serves requests. Pick one option:

Option	Who runs it	How to get it	Best for
Serverless	Kong	Created automatically with a Serverless gateway	Quick start; 100 req/s, 10 MB body limit, public only
Dedicated Cloud Gateway	Kong	Gateway → Data plane nodes → configure cloud data plane; pick region and size	Production, private networking; may need a paid plan
Self-hosted	You (Docker / Kubernetes)	Gateway → Data plane nodes → New data plane node → copy generated command	Local testing, full control

Serverless (Path A — used here)

1. Nothing to install. After Step 1, wait about 30 seconds.
2. Verify the proxy hostname resolves: `nslookup 4b1ed59e0a.us.serverless.gateways.konggateway.com` must return IP addresses.
3. `curl https://<proxy-url>/` should return Kong's `no Route matched` JSON. That proves the data plane is live.

Self-hosted Docker (if Serverless is unavailable)

1. Gateway → **Data plane nodes** → **New data plane node** → platform **Docker** (or macOS).
2. Click **Generate certificate**, then copy the generated `docker run` command. It contains the control plane endpoints and certs. It looks like this:

```
docker run -d --name kong-dp -p 8000:8000 -p 8443:8443 \
  -e KONG_ROLE=data_plane -e KONG_DATABASE=off -e KONG_KONNECT_MODE=on \
  -e KONG_CLUSTER_MTLS=pki \
  -e KONG_CLUSTER_CONTROL_PLANE=<id>.us.cp.konghq.com:443 \
  -e KONG_CLUSTER_SERVER_NAME=<id>.us.cp.konghq.com \
  -e KONG_CLUSTER_TELEMETRY_ENDPOINT=<id>.us.tp.konghq.com:443 \
  -e KONG_CLUSTER_TELEMETRY_SERVER_NAME=<id>.us.tp.konghq.com \
  -e KONG_CLUSTER_CERT="<generated cert>" -e KONG_CLUSTER_CERT_KEY="
<generated key>" \
  -e KONG_LUA_SSL_TRUSTED_CERTIFICATE=system \
  <image shown in Konnect>
```

3. Run it, then check **Data plane nodes** shows the node as **Connected**.
4. Your proxy URL is `http://localhost:8000`.

This cert-based connection is the mTLS that the control plane demands; it is why plain `curl` to `*.cp.konghq.com` fails.

Path B note: the Serverless AI Gateway in this account never got a data plane. Its edge DNS `b9d624f9a5.gateways.konggateway.com` returns NXDOMAIN and Data plane nodes is empty. Raise it with Kong support (chat bubble in Konnect) quoting gateway ID `35d895bb-08ce-48c3-ae48-00ffaea68dc1`, or attach a self-hosted data plane.

Step 3A — AI Proxy on the Serverless API Gateway (working setup)

Three entities on `serverless-api-gateway-demo` route `/chat/completions` to Azure. This is the setup that returned HTTP 200 from `gpt-5.4-mini`.

1. Gateway Service

Gateway → **Gateway services** → **New gateway service** → Full URL:

Field	Value
Full URL	`https://srinipolagani-8291-resource.openai.azure.com`
Name	`azure-foundry`

AI Proxy builds the real Azure path, but the Service still needs a URL.

2. Route

Gateway → **Routes** → **New route** → Basic:

Field	Value
Name	`azure-chat`
Gateway service	`azure-foundry`
Path	`/chat/completions`
Strip path	on (default)

3. AI Proxy plugin

Route `azure-chat` → **Plugins** → **New plugin** → **AI Proxy** (scoped to the route):

Section	Field	Value
Plugin configuration	Route type	`llm/v1/chat`
Model	Provider	`azure`
Model	Name	`gpt-5.4-mini`
Model → Options (toggle on)	Azure instance	`srinipolagani-8291-resource`
Model → Options	Azure API version	`2025-04-01-preview` (replace default `2023-05-15`)
Model → Options	Azure deployment ID	`gpt-5.4-mini`
Show additional settings → Auth (toggle on)	Header name	`api-key`
Auth	Header value	your Azure key (no `Bearer` prefix)

Click **Save** at the bottom. The Konnect chat bubble can cover the button; close it first. The page must switch to the Plugins list showing **AI Proxy · Route · Enabled**; if it stays on "Create Plugin", it was not saved.

For the Responses API, add a second Route with path `/responses` and a second AI Proxy plugin with route type `llm/v1/responses`.

Step 3B — AI Gateway 2.0 (Model Provider + AI Model)

This is the newer model-centric UI. The config below is saved in this account as model `gpt-foundry` → `azure-foundry: gpt-5.4-mini`; it only serves traffic once the AI Gateway has a working data plane (Step 2).

1. Model Provider — AI Gateway → **Providers** → New

Field	Value
Provider	Azure
Service	Azure OpenAI (or Azure AI Foundry if listed)
Name	`azure-foundry`
Azure instance	`srinipolagani-8291-resource`
Authentication	Provided by AI gateway
Authentication method	API key
Key source	Enter a key (or Token Vault for production)
Header name	`api-key`
API key	your Azure key
Parameter name / value / location	leave blank

2. AI Model — AI Gateway → **Models** → New

Form step	Field	Value
1. General information	Display name	`gpt-foundry` (clients send this as `model`)
2. Target models	Provider	`azure-foundry`
2. Target models	Target model	`gpt-5.4-mini`
2. Target models → Advanced fields	Azure deployment ID	`gpt-5.4-mini` (required; else `must set 'model.options.azure_deployment_id'`)
2. Target models → Advanced fields	Azure API version	`2025-04-01-preview`

Form step	Field	Value
3. Capabilities	Enable	Generate (`/chat/completions`), Agentic (`/responses`); format OpenAI
4. Routing	Base path	blank or e.g. `/azure`
4. Routing	Model alias / match location	defaults
5. Authentication and access	Auth strategy	API key (or none while testing)
6. Advanced configuration	—	defaults

3. Call it — the model's **Connect** tab shows a template with `<YOUR_DP_HOST>`. Replace it with the AI Gateway's Public edge DNS (Step 1B), not the Admin API address:

```
curl -X POST "https://<AI-gateway-edge-DNS>/chat/completions" \
  -H 'Content-Type: application/json' \
  -d '{"model":"gpt-foundry","messages":[{"role":"user","content":"Hello"}]}'
```

Step 4 — Same setup as code with decK

decK applies Step 3A from a file, so the setup is repeatable and versioned. Save as `kong.yaml`:

```
_format_version: "3.0"

services:
  - name: azure-foundry
    url: https://srinipolagani-8291-resource.openai.azure.com
    routes:
      - name: azure-chat
        paths:
          - /chat/completions
        strip_path: true
        plugins:
          - name: ai-proxy
            config:
              route_type: llm/v1/chat
```

```
      auth:
        header_name: api-key
        header_value: ${{ env "DECK_AZURE_KEY" }}
      model:
        provider: azure
        name: gpt-5.4-mini
        options:
          azure_instance: srinipolagani-8291-resource
          azure_deployment_id: gpt-5.4-mini
          azure_api_version: "2025-04-01-preview"
```

Set credentials (decK only substitutes variables starting with `DECK_`):

```
export KONNECT_TOKEN="kpat_..."
export DECK_AZURE_KEY="<Azure key>"
```

Preview, then apply:

```
deck gateway diff kong.yaml \
  --konnect-addr https://us.api.konghq.com \
  --konnect-token $KONNECT_TOKEN \
  --konnect-control-plane-name serverless-api-gateway-demo

deck gateway sync kong.yaml \
  --konnect-addr https://us.api.konghq.com \
  --konnect-token $KONNECT_TOKEN \
  --konnect-control-plane-name serverless-api-gateway-demo
```

- `sync` makes Konnect match the file exactly and deletes anything not in it. Always `diff` first.
- Export what is live: `deck gateway dump -o current.yaml` with the same flags.
- A clean re-run of `diff` shows 0 creates, 0 updates, 0 deletes.

Step 4b — Automate everything with Terraform

One `terraform apply` can create the Azure resource and deployment, then wire Kong to it using the key Azure returns, so no key is ever typed. Use either Terraform or decK for the Kong entities, never both.

Layer	Terraform provider	Resources
Azure Foundry resource + deployment	`hashicorp/azurerm`	`azurerm_cognitive_account`, `azurerm_cognitive_deployment`
Kong control plane	`kong/konnect`	existing one passed by ID (or the Serverless Cloud Gateway resource)
Service, Route, AI Proxy	`kong/konnect`	`konnect_gateway_service`, `konnect_gateway_route`, `konnect_gateway_plugin_ai_proxy`
Key Auth + consumer	`kong/konnect`	`konnect_gateway_plugin_key_auth`, `konnect_gateway_consumer`, `konnect_gateway_key_auth`
AI Gateway 2.0 models/providers	none found	use `kongctl` or the Konnect API

`main.tf`:

```
terraform {
  required_providers {
    konnect = { source = "kong/konnect" }
    azurerm = { source = "hashicorp/azurerm" }
  }
}

provider "konnect" {
  server_url = "https://us.api.konghq.com" # token from env KONNECT_TOKEN
}

provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
}

variable "subscription_id" { type = string }
variable "resource_group"  { type = string }
variable "location"        { type = string }
variable "control_plane_id" {
  type    = string
  default = "5e42f6e0-73d1-4f2e-a9dc-bd210184514e" # serverless-api-gateway-
```

```
demo
}

# ---------- Azure AI Foundry ----------
resource "azurerm_cognitive_account" "foundry" {
  name                = "srinipolagani-8291-resource"
  location            = var.location
  resource_group_name = var.resource_group
  kind                = "AIServices"
  sku_name            = "S0"
  custom_subdomain_name = "srinipolagani-8291-resource"
}

resource "azurerm_cognitive_deployment" "gpt" {
  name                 = "gpt-5.4-mini"
  cognitive_account_id = azurerm_cognitive_account.foundry.id
  model {
    format  = "OpenAI"
    name    = "gpt-5.4-mini"
    version = "2026-03-17"
  }
  sku {
    name     = "GlobalStandard"
    capacity = 150
  }
}

# ---------- Kong Konnect ----------
resource "konnect_gateway_service" "azure" {
  control_plane_id = var.control_plane_id
  name             = "azure-foundry"
  protocol         = "https"
  host             = "srinipolagani-8291-resource.openai.azure.com"
  port             = 443
}

resource "konnect_gateway_route" "chat" {
  control_plane_id = var.control_plane_id
  name             = "azure-chat"
  paths            = ["/chat/completions"]
  methods          = ["POST"]
  strip_path       = true
  service = { id = konnect_gateway_service.azure.id }
```

```
}

resource "konnect_gateway_plugin_ai_proxy" "azure" {
  control_plane_id = var.control_plane_id
  enabled          = true
  route = { id = konnect_gateway_route.chat.id }
  config = {
    route_type = "llm/v1/chat"
    auth = {
      header_name  = "api-key"
      header_value = azurerm_cognitive_account.foundry.primary_access_key
    }
    model = {
      provider = "azure"
      name     = azurerm_cognitive_deployment.gpt.name
      options = {
        azure_instance      =
azurerm_cognitive_account.foundry.custom_subdomain_name
        azure_deployment_id = azurerm_cognitive_deployment.gpt.name
        azure_api_version   = "2025-04-01-preview"
      }
    }
  }
}

# ---------- Lock it down ----------
resource "konnect_gateway_plugin_key_auth" "chat" {
  control_plane_id = var.control_plane_id
  route = { id = konnect_gateway_route.chat.id }
  config = { key_names = ["apikey"] }
}

resource "konnect_gateway_consumer" "app" {
  control_plane_id = var.control_plane_id
  username         = "my-app"
}

resource "konnect_gateway_key_auth" "app" {
  control_plane_id = var.control_plane_id
  consumer_id      = konnect_gateway_consumer.app.id
}
```

```
output "app_api_key" {
  value     = konnect_gateway_key_auth.app.key
  sensitive = true
}
```

Run it:

```
export KONNECT_TOKEN="kpat_..."
az login
terraform init
terraform plan  -var subscription_id=<sub> -var resource_group=<rg> -var
location=<region>
terraform apply -var subscription_id=<sub> -var resource_group=<rg> -var
location=<region>
terraform output -raw app_api_key   # give this to the client app
```

Adopting what already exists. The Azure resource, deployment and Kong entities were created by hand. Either import them, or delete the Kong entities in the UI first so Terraform can create them:

```
terraform import azurerm_cognitive_account.foundry \

/subscriptions/<sub>/resourceGroups/<rg>/providers/Microsoft.CognitiveService
s/accounts/srinipolagani-8291-resource
terraform import azurerm_cognitive_deployment.gpt \

/subscriptions/<sub>/resourceGroups/<rg>/providers/Microsoft.CognitiveService
s/accounts/srinipolagani-8291-resource/deployments/gpt-5.4-mini
```

Before you rely on it

- Run `terraform validate` first. The Konnect attribute names follow Kong's published examples, but pin the provider version and check the registry docs for that version.
- The Azure key and the consumer key land in Terraform state. Use an encrypted remote backend (Azure Storage, Terraform Cloud), never a local state file in git.
- Serverless control planes can also be created in Terraform (the provider lists a Serverless Cloud Gateway resource); passing the existing ID is simpler and avoids recreating the proxy URL.
- AI Gateway 2.0 (Step 3B) models and providers had no Terraform resources in Kong's docs; manage them with `kongctl` or the Konnect API until they do.

- Rotating the key: `az cognitiveservices account keys regenerate ...`, then `terraform apply` updates the plugin automatically.

Step 5 — Test

A working call returns HTTP 200, a header `x-kong-llm-model: azure/gpt-5.4-mini`, and a reply in `choices[0].message.content`. Wait about 15 seconds after any config change.

1. Data plane is up (expect `no Route matched` before Step 3):

```
curl -i https://4b1ed59e0a.us.serverless.gateways.konggateway.com/
```

2. Chat completion through Kong:

```
curl -i -X POST
"https://4b1ed59e0a.us.serverless.gateways.konggateway.com/chat/completions"
\
  -H 'Content-Type: application/json' \
  -d '{"messages":[{"role":"user","content":"How does Kong AI Gateway
work?"}]}'
```

3. Just the answer:

```
curl -s -X POST
"https://4b1ed59e0a.us.serverless.gateways.konggateway.com/chat/completions"
\
  -H 'Content-Type: application/json' \
  -d '{"messages":[{"role":"user","content":"Say hello in 5 words"}]}' \
  | jq -r '.choices[0].message.content'
```

4. After adding Key Auth (Step 6), add `-H 'apikey: <consumer-key>'`; without it you should get 401.
5. Konnect → gateway → **Analytics** shows the requests within a minute or two.

Header to check	Meaning
`x-kong-llm-model: azure/gpt-5.4-mini`	AI Proxy handled the request
`x-ms-served-model`	The Azure model version that answered
`x-ratelimit-remaining-requests`	Azure quota left this minute (limit 150)

Header to check	Meaning
`x-kong-upstream-latency`	Time spent in Azure, ms

Step 6 — Secure and harden

The route is open today: anyone with the URL spends your Azure quota. Do these before sharing it.

- [ ] **Rotate the Azure key** that was shared in chat: Azure Portal → Keys and Endpoint → Regenerate Key 1, then update the plugin's Header value (or `DECK_AZURE_KEY` and re-sync).
- [ ] **Add Key Auth**: Route `azure-chat` → Plugins → **Key Auth**. Then **Consumers** → New consumer → Credentials → Key Auth → generate a key. Clients send `apikey: <key>`.
- [ ] **Rate limit** under Azure's 150 requests/min: add **Rate Limiting** (e.g. 100/minute) or **AI Rate Limiting Advanced** for token-based limits.
- [ ] **Store the key in a vault**: Konnect Config Store / vault, then reference it in Header value instead of plain text.
- [ ] **Restrict methods** on the route to `POST`.
- [ ] **Turn on Log statistics** in AI Proxy to see token usage and cost in Analytics.

Troubleshooting

Every error below was hit during this setup; the fix is in the right column.

Error	Cause	Fix
`mTLS headers missing` / `missing headers in request`	Called the control plane (Admin API, `*.cp.konghq.com`)	Call the proxy URL / Public edge DNS instead
`getaddrinfo ENOTFOUND <id>.gateways.konggateway.com`	AI Gateway has no data plane; DNS name never created	Check Data plane nodes; ask Kong support or attach a self-hosted data plane

Error	Cause	Fix
`Client network socket disconnected before secure TLS connection`	Wildcard host resolves but no gateway serves that ID	Same as above
`must set 'model.options.azure_deployment_id' for azure provider`	Deployment ID missing on the model target	Target → Advanced fields → Azure deployment ID = `gpt-5.4-mini`
`no Route matched with those values`	Data plane live, but no Route for that path	Create the Route (Step 3A)
HTTP 200 with empty body, no `x-kong-llm-model` header	AI Proxy plugin not saved; request went to Azure root	Save the plugin; confirm it shows in the Plugins list
401 from Azure	Wrong or empty key, or wrong header	Header `api-key` + raw key, or `Authorization` + `Bearer <key>`; check `DECK_AZURE_KEY` length
404 `DeploymentNotFound`	Typo in instance or deployment ID	Match Foundry exactly
429 from Azure	Over 150 requests/min on the deployment	Add a rate limit in Kong or raise Azure quota
Entities "disappear" or a page logs you out	Two Konnect organizations; resources live in one	Use the org switcher; build everything in one org